

Lab C

Counter & Timer



课程名称：数字逻辑电路设计

作者：王传宇 3250102681

专业：计算机科学与技术

邮箱：3250102681@zju.edu.cn

时间：2026-5-28

地点：紫金港东四 509 室

第一部分：操作方法与实验步骤

任务一：实现 **74LS161** 功能

1.1.1. 模块 **My74LS161** 代码

根据 74LS161 的功能表，在给定的代码框架中实现同步计数、异步清零、同步置数和进位逻辑。

```
module My74LS161(  
    input CP,  
    input CRn,  
    input LDn,  
    input [3:0] D,  
    input CTT,  
    input CTP,  
    output [3:0] Q,  
    output CO  
);  
    reg [3:0] Q_reg = 4'b0;  
  
    always @(posedge CP or negedge CRn) begin  
        if (!CRn) begin  
            Q_reg <= 4'b0;           // Asynchronous clear  
        end else if (!LDn) begin  
            Q_reg <= D;             // Synchronous load  
        end else if (CTT && CTP) begin  
            Q_reg <= Q_reg + 1'b1;  // Count up  
        end  
    end  
  
    assign Q = Q_reg;  
    assign CO = (Q == 4'hF);       // Carry out when all ones  
endmodule
```

1.1.2. 仿真代码

编写 `tb_My74LS161.v`，覆盖异步清零、同步置数、计数使能、进位输出及优先级测试。

```
`timescale 1ns / 1ps  
  
module tb_My74LS161();
```

```

reg CP;
reg CRn;
reg LDn;
reg [3:0] D;
reg CTT;
reg CTP;
wire [3:0] Q;
wire CO;

My74LS161 uut (
    .CP(CP), .CRn(CRn), .LDn(LDn), .D(D),
    .CTT(CTT), .CTP(CTP), .Q(Q), .CO(CO)
);

always #5 CP = ~CP; // 100 MHz clock

initial begin
    CP = 0; CRn = 1; LDn = 1; D = 4'b0; CTT = 0; CTP = 0;

    // Async clear
    #10; CRn = 0; #10; CRn = 1; #10;
    $display("After async clear: Q = %h, CO = %b", Q, CO);

    // Synchronous load
    D = 4'b0101; LDn = 0; #10; LDn = 1; #10;
    $display("After load: Q = %h, CO = %b", Q, CO);

    // Count enable
    CTT = 1; CTP = 1;
    repeat(6) #10;
    $display("After 6 counts from 5: Q = %h, CO = %b", Q, CO);

    // Hold by deasserting CTT
    CTT = 0;
    repeat(3) #10;
    $display("After hold (CTT=0): Q = %h, CO = %b", Q, CO);

    // Count to overflow
    CTT = 1;
    repeat(6) #10; // Continue: B, C, D, E, F, 0
    #10;
    $display("After rollover: Q = %h, CO = %b", Q, CO);

    // Carry out test

```

```

    CRn = 0; #10; CRn = 1;
    D = 4'b1111; LDn = 0; #10; LDn = 1; #10;
    $display("At Q=F: Q = %h, CO = %b (should be 1)", Q, CO);

    // Async clear priority
    CRn = 0; CTT = 1; CTP = 1;
    #5;
    $display("Mid-cycle clear: Q = %h (should be 0)", Q);
    CRn = 1; #5;
    $display("After clear release: Q = %h", Q);

    $finish;
end
endmodule

```

任务二：74LS161 应用（时钟）

1.2.1. 顶层模块 top 代码

使用四个 My74LS161 级联实现 24 小时制的“小时:分钟”时钟，支持快慢时钟选择和复位至 23:00。

```

module top(
    input clk,
    input [1:0] SW,
    output [3:0] AN,
    output [7:0] SEGMENT
);
    wire clk_10ms, clk_100ms;
    clk_10ms clk_div_10ms (.clk(clk), .clk_10ms(clk_10ms));
    clk_100ms clk_div_100ms (.clk(clk), .clk_100ms(clk_100ms));

    wire clk_counter = (SW[0] == 1'b0) ? clk_10ms : clk_100ms;

    wire [3:0] hour_tens, hour_ones, min_tens, min_ones;

    // Terminal count indicators
    wire min_ones_tc = (min_ones == 4'd9);
    wire min_tens_tc = (min_tens == 4'd5) & min_ones_tc; // 59
    wire hour_ones_tc = (hour_tens == 4'd2) ? (hour_ones == 4'd3) : (hour_ones == 4'd9);
    wire hour_tens_tc = (hour_tens == 4'd2) & hour_ones_tc & min_tens_tc;

    // LDn signals (low active) - reset at boundaries or when SW[1] active

```

```

wire min_ones_ldn = ~(min_ones_tc | SW[1]);
wire min_tens_ldn = ~(min_tens_tc | SW[1]);
wire hour_ones_ldn = ~(hour_ones_tc & min_tens_tc | SW[1]);
wire hour_tens_ldn = ~(hour_tens_tc | SW[1]);

// D inputs: during reset load 23:00, otherwise zero at boundaries
wire [3:0] min_ones_D = 4'd0;
wire [3:0] min_tens_D = 4'd0;
wire [3:0] hour_ones_D = SW[1] ? 4'd3 : 4'd0;
wire [3:0] hour_tens_D = SW[1] ? 4'd2 : 4'd0;

// Count enables for each digit
wire min_ones_ctt = 1'b1;
wire min_tens_ctt = min_ones_tc;
wire hour_ones_ctt = min_tens_tc;
wire hour_tens_ctt = hour_ones_tc & min_tens_tc;

My74LS161 u_min_ones (
    .CP(clk_counter), .CRn(1'b1), .LDn(min_ones_ldn),
    .D(min_ones_D), .CTT(min_ones_ctt), .CTP(1'b1),
    .Q(min_ones), .CO()
);

My74LS161 u_min_tens (
    .CP(clk_counter), .CRn(1'b1), .LDn(min_tens_ldn),
    .D(min_tens_D), .CTT(min_tens_ctt), .CTP(1'b1),
    .Q(min_tens), .CO()
);

My74LS161 u_hour_ones (
    .CP(clk_counter), .CRn(1'b1), .LDn(hour_ones_ldn),
    .D(hour_ones_D), .CTT(hour_ones_ctt), .CTP(1'b1),
    .Q(hour_ones), .CO()
);

My74LS161 u_hour_tens (
    .CP(clk_counter), .CRn(1'b1), .LDn(hour_tens_ldn),
    .D(hour_tens_D), .CTT(hour_tens_ctt), .CTP(1'b1),
    .Q(hour_tens), .CO()
);

// Display
DisplayNumber display_inst (
    .clk(clk), .rst(1'b0), .hexs({hour_tens, hour_ones, min_tens,
min_ones}),
    .points(4'b0100), .LEs(4'b0000), .AN(AN), .SEGMENT(SEGMENT)
);

```

```
);  
endmodule
```

1.2.2. 代码解释

- 时钟选择：通过 SW[0] 在 10 ms 和 100 ms 时钟源之间切换，方便调试和正常走时观察。所有 My74LS161 的 CP 均接自 clk_counter。
- 终端计数：为每位定义了 tc 信号，如 min_ones_tc 当分钟个位为 9 时有效。这些信号用于使能下一位计数、触发自身同步置零，并组合产生各模块的 LDn。
- 同步置数：每位到达最大计数值（如个位 9、十位 5、小时个位在 0~9 或 0~3 根据十位值）时，下一个时钟沿自动置零（LDn=0, D=0）。复位时 SW[1] 强制所有位加载预设值，实现 23:00 的初始化。
- 级联使能：通过 CTT 控制高位的计数权限，例如 min_tens_ctt = min_ones_tc，仅当个位为 9 且自身计数时，十位才能加 1。
- 显示：四个 4 位值拼成 16 位 hex 总线送入 DisplayNumber 动态扫描驱动四位共阳极数码管，中间小数点常亮以区分小时与分钟。

第二部分：实验结果与分析

74LS161 仿真波形

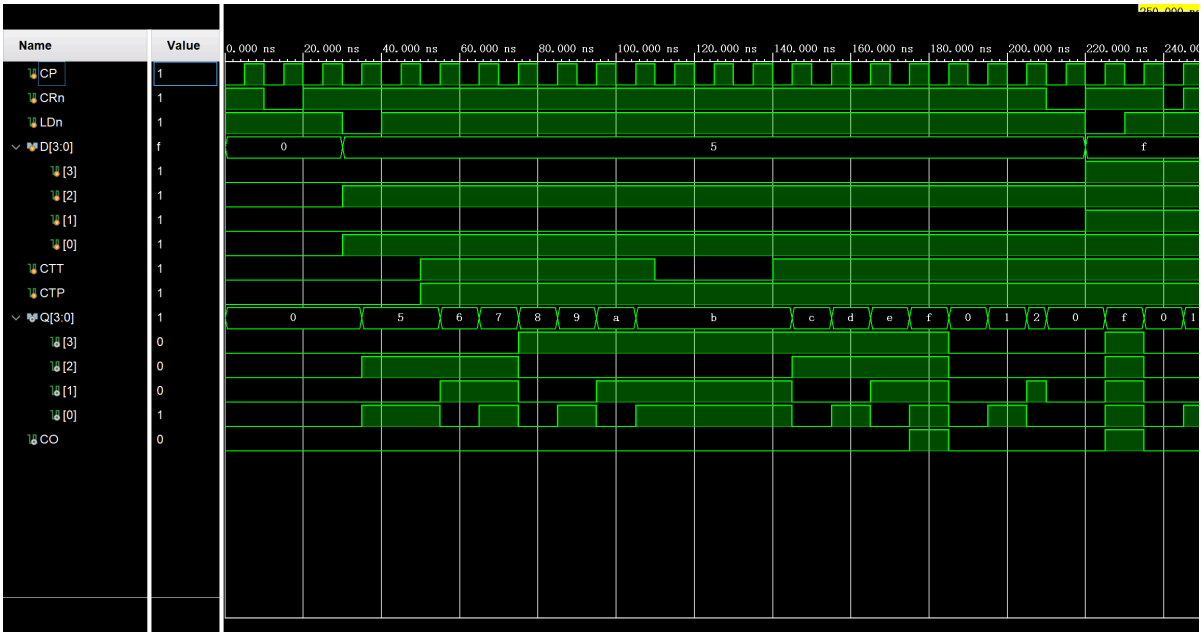


Figure 1: My74LS161 仿真波形

波形分析：

- 异步清零：CRn 置低后 Q 立即变为 0，不受时钟控制；释放后下一个时钟才可能更新。
- 同步置数：在 LDn=0 时，时钟上升沿将 D (5) 加载到 Q，之后 LDn 恢复高，计数器由 5 开始递增。
- 计数使能：CTT 和 CTP 同为高时，每个时钟上升沿 Q 加 1；当 CTT 拉低后，计数暂停，值保持。
- 进位输出：当 Q=1111 时 CO 立即为高，与时钟无关；计数回 0 后 CO 变低。
- 优先级：异步清零在所有控制中优先级最高。波形完整验证了 74LS161 的所有功能。

时钟应用下板验证

下板照片演示了时钟在三种条件下的工作状态。

1. 初始化至 23:00:

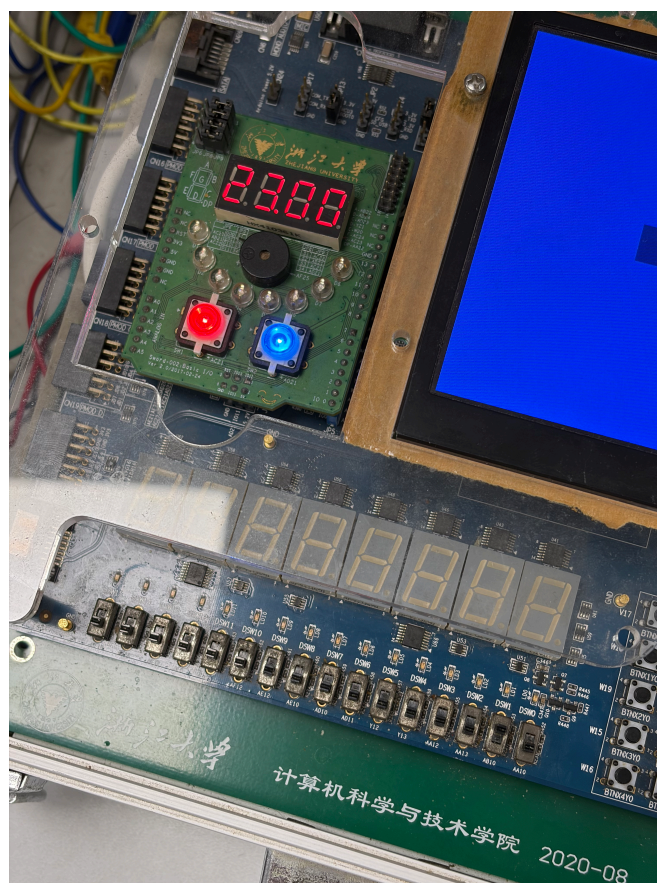


Figure 2: SW[1]=1 时复位至 23:00

当 SW[1] 拨至高时，小时位被置为 23，分钟位置为 00，数码管稳定显示 23.00，中间小数点被点亮。

2. 快速时钟模式 (10 ms):

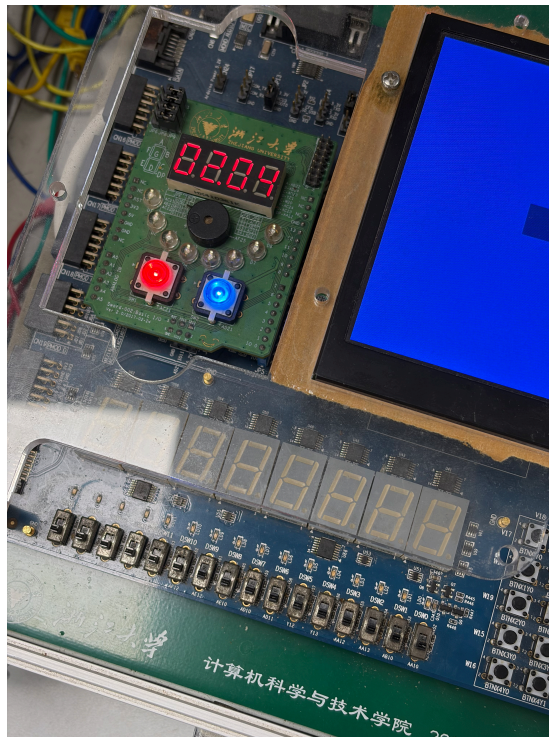


Figure 3: SW[0]=0 时分钟快速递增

SW[0]=0 时选择 10 ms 时钟, 分钟十位和个位快速翻转, 从 23:00 迅速递增。数码管刷新流畅, 无闪烁。

3. 慢速时钟模式 (100 ms):

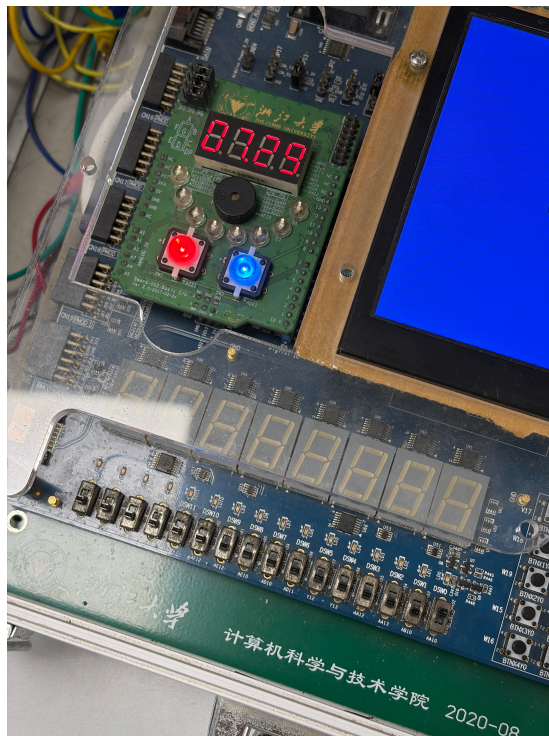


Figure 4: SW[0]=1 时分钟以 100 ms 间隔递增

SW[0]=1 时使用 100 ms 时钟，分钟变化明显减慢，可清晰观察从 59 到 00 的进位过程以及小时位的正确跳变（例如 23 归零）。

所有测试中，时钟走时逻辑正确，级联进位与复位功能正常，数码管显示稳定，证明了 74LS161 应用模块的有效性。

第三部分：讨论与心得

无。

附录 1：修改之后的 **DisplayNumber.v**

```
module DisplayNumber(  
    input wire      clk,  
    input wire      rst,          // synchronous reset, active high  
    input wire [15:0] hexs,       // 4 digits, each 4 bits: {digit3, digit2,  
    digit1, digit0}  
    input wire [3:0] points,      // decimal point control, 1 = light  
    input wire [3:0] LEs,        // latch enable (low active), not used here  
    output reg [3:0] AN,         // anode select, low active  
    output reg [7:0] SEGMENT      // segment outputs, SEGMENT[7]=DP  
);  
  
// Internal scan clock (about 1kHz from 100MHz)  
reg [16:0] scan_cnt;  
always @(posedge clk or posedge rst) begin  
    if (rst)  
        scan_cnt <= 17'd0;  
    else  
        scan_cnt <= scan_cnt + 1;  
end  
  
// Scan state: select which digit to drive (0~3)  
wire [1:0] scan_sel = scan_cnt[16:15]; // upper bits toggles slowly  
  
// Current digit data  
reg [3:0] current_hex;  
reg      current_point;  
always @* begin  
    case (scan_sel)  
        2'b00: begin
```

```

        current_hex  = hexs[3:0];    // rightmost digit (min_ones)
        current_point = points[0];
    end
    2'b01: begin
        current_hex  = hexs[7:4];    // min_tens
        current_point = points[1];
    end
    2'b10: begin
        current_hex  = hexs[11:8];   // hour_ones
        current_point = points[2];
    end
    2'b11: begin
        current_hex  = hexs[15:12];  // hour_tens
        current_point = points[3];
    end
    default: begin
        current_hex  = 4'b0;
        current_point = 1'b0;
    end
endcase
end

// Anode control (low active) - enable only the selected digit
always @* begin
    AN = 4'b1111;                // all off
    AN[scan_sel] = 1'b0;
end

reg [6:0] seg_low; // {g,f,e,d,c,b,a}
always @* begin
    case (current_hex)
        4'h0: seg_low = 7'b1000000; // 0
        4'h1: seg_low = 7'b1111001; // 1
        4'h2: seg_low = 7'b0100100; // 2
        4'h3: seg_low = 7'b0110000; // 3
        4'h4: seg_low = 7'b0011001; // 4
        4'h5: seg_low = 7'b0010010; // 5
        4'h6: seg_low = 7'b0000010; // 6
        4'h7: seg_low = 7'b1111000; // 7
        4'h8: seg_low = 7'b0000000; // 8
        4'h9: seg_low = 7'b0010000; // 9
        4'hA: seg_low = 7'b0001000; // A
        4'hB: seg_low = 7'b0000011; // b
        4'hC: seg_low = 7'b1000110; // C
    endcase
end

```

```

        4'hD: seg_low = 7'b0100001; // d
        4'hE: seg_low = 7'b0000110; // E
        4'hF: seg_low = 7'b0001110; // F
        default: seg_low = 7'b1111111;

    endcase

end

// Combine segments and decimal point (active low for dp as well)
// SEGMENT = {dp, g, f, e, d, c, b, a}
always @* begin
    SEGMENT = {~current_point, seg_low}; // dp is lit when current_point=1,
so output 0
end

endmodule

```

附录2：约束文件

```

# Filename: constraints.xdc
# Constraints for LabC Clock Application

# ---- Main clock ----
set_property PACKAGE_PIN AC18 [get_ports clk]
set_property IOSTANDARD LVCMOS18 [get_ports clk]
create_clock -period 10.000 -name clk [get_ports clk]

# ---- Slide switches ----
set_property PACKAGE_PIN AA10 [get_ports {SW[0]}]
set_property PACKAGE_PIN AB10 [get_ports {SW[1]}]
set_property IOSTANDARD LVCMOS15 [get_ports {SW[0]}]
set_property IOSTANDARD LVCMOS15 [get_ports {SW[1]}]

# ---- 7-segment display anodes ----
set_property PACKAGE_PIN AD21 [get_ports {AN[0]}]
set_property PACKAGE_PIN AC21 [get_ports {AN[1]}]
set_property PACKAGE_PIN AB21 [get_ports {AN[2]}]
set_property PACKAGE_PIN AC22 [get_ports {AN[3]}]
set_property IOSTANDARD LVCMOS33 [get_ports {AN[0]}]
set_property IOSTANDARD LVCMOS33 [get_ports {AN[1]}]
set_property IOSTANDARD LVCMOS33 [get_ports {AN[2]}]
set_property IOSTANDARD LVCMOS33 [get_ports {AN[3]}]

```

```
# ---- 7-segment segments ----  
set_property PACKAGE_PIN AB22 [get_ports {SEGMENT[0]}]  
set_property PACKAGE_PIN AD24 [get_ports {SEGMENT[1]}]  
set_property PACKAGE_PIN AD23 [get_ports {SEGMENT[2]}]  
set_property PACKAGE_PIN Y21  [get_ports {SEGMENT[3]}]  
set_property PACKAGE_PIN W20  [get_ports {SEGMENT[4]}]  
set_property PACKAGE_PIN AC24 [get_ports {SEGMENT[5]}]  
set_property PACKAGE_PIN AC23 [get_ports {SEGMENT[6]}]  
set_property PACKAGE_PIN AA22 [get_ports {SEGMENT[7]}]  
set_property IOSTANDARD LVCMOS33 [get_ports {SEGMENT[0]}]  
set_property IOSTANDARD LVCMOS33 [get_ports {SEGMENT[1]}]  
set_property IOSTANDARD LVCMOS33 [get_ports {SEGMENT[2]}]  
set_property IOSTANDARD LVCMOS33 [get_ports {SEGMENT[3]}]  
set_property IOSTANDARD LVCMOS33 [get_ports {SEGMENT[4]}]  
set_property IOSTANDARD LVCMOS33 [get_ports {SEGMENT[5]}]  
set_property IOSTANDARD LVCMOS33 [get_ports {SEGMENT[6]}]  
set_property IOSTANDARD LVCMOS33 [get_ports {SEGMENT[7]}]
```